String Algorithms

for Competitive Programming

A Pattern-Recognition Primer — the prereq before going expert

October 22, 2026

Abstract

Strings are everywhere in competitive programming: from the trivial "count occurrences" to "find the number of distinct substrings of every prefix" or "compute the smallest cyclic shift of a 10^6-character text". The bottleneck is rarely *which* algorithm to code — once you know KMP you can write it from memory. The hard part is *recognising* that a problem is a Z-problem, or a Suffix-Automaton problem, or an Aho–Corasick problem in disguise. This document is built around that recognition step. Each chapter follows the same structure: concept, worked example, code, complexity, and a dedicated `When to use` block that summarises the problem signatures that should make you reach for that algorithm. Everything here compiles with stock `pdflatex`; C++ snippets are inline so you can paste them straight into a Codeforces submission.

Contents

1 Foundations & Pattern-Recognition Framework **4**
- 1.1 Notation . . . 4
- 1.2 Complexity conventions . . . 4
- 1.3 The pattern-recognition framework . . . 4
- 1.4 A note on C++ . . . 5

2 Polynomial String Hashing & Rabin–Karp **5**
- 2.1 Definition . . . 5
- 2.2 Implementation . . . 5
- 2.3 Rabin–Karp: pattern matching with hashing . . . 6
- 2.4 Worked example . . . 7
- 2.5 When to use hashing . . . 7
- 2.6 Anti-hash: how authors break you . . . 8
- 2.7 Codeforces references . . . 8

3 Prefix Function & KMP **8**
- 3.1 Definition . . . 8
- 3.2 Linear computation . . . 9
- 3.3 KMP search . . . 9
- 3.4 Borders and periods . . . 9
- 3.5 Failure automaton . . . 10
- 3.6 Worked example . . . 10
- 3.7 When to use the prefix function . . . 10
- 3.8 Codeforces references . . . 10

4 Z-Function **11**
4.1 Definition . . . 11
4.2 Comparison to the prefix function . . . 11
4.3 Linear computation . . . 11
4.4 Pattern matching with Z . . . 12
4.5 Worked example . . . 12
4.6 When to use the Z-function . . . 12
4.7 Codeforces references . . . 12

5 Trie (Prefix Tree) **13**
5.1 Definition and operations . . . 13
5.2 Complexity . . . 14
5.3 Worked example . . . 14
5.4 When to use a trie . . . 14
5.5 Codeforces references . . . 15

6 Aho–Corasick Automaton **15**
6.1 Idea . . . 15
6.2 Failure links . . . 15
6.3 Implementation . . . 15
6.4 Complexity . . . 16
6.5 Worked example . . . 16
6.6 When to use Aho–Corasick . . . 17
6.7 Codeforces references . . . 17

7 Suffix Array & LCP **18**
7.1 Definition . . . 18
7.2 Construction: $\mathcal{O}(n \log^2 n)$ via sort by rank pairs . . . 18
7.3 LCP: Kasai's algorithm . . . 19
7.4 Sparse table for RMQ on LCP . . . 19
7.5 Distinct substrings . . . 20
7.6 Worked example . . . 20
7.7 When to use the suffix array . . . 20
7.8 Codeforces references . . . 21

8 Suffix Automaton **21**
8.1 Core idea: endpos equivalence . . . 21
8.2 Online construction . . . 21
8.3 Standard applications . . . 22
8.4 Worked example . . . 22
8.5 When to use the suffix automaton . . . 23
8.6 Codeforces references . . . 23

9 Palindromes: Manacher & Eertree **23**
9.1 Manacher's algorithm . . . 23
9.2 Eertree (palindromic tree) . . . 24
9.3 Worked example . . . 25
9.4 When to use Manacher / eertree . . . 25
9.5 Codeforces references . . . 26

10 Trie Tricks: XOR, Persistent, Merge **26**
10.1 Binary (XOR) trie . . . 26
10.2 Persistent trie . . . 27
10.3 Small-to-large trie merge . . . 27
10.4 When to use trie tricks . . . 28
10.5 Codeforces references . . . 28

11 Advanced: Lyndon, Duval, Booth **28**
11.1 Lyndon words and Duval's factorisation . . . 28
11.2 Booth's algorithm . . . 29
11.3 Runs (brief mention) . . . 29
11.4 When to use Lyndon / Duval / Booth . . . 30
11.5 Codeforces references . . . 30

12 Pattern Recognition Cheat Sheet **30**
12.1 Decision table . . . 30
12.2 Decision flowchart (in words) . . . 31
12.3 A note on hybrid problems . . . 31

References & Further Reading **32**

1 Foundations & Pattern-Recognition Framework

Before collecting algorithms we need a shared vocabulary. Most string bugs in CP are not algorithmic — they are *indexing* bugs, where a writer off-by-one shifts a substring range by a single character and the answer is silently wrong. Treat this chapter as a contract you sign with yourself for the rest of the document.

1.1 Notation

Throughout this document we write S for a string, $n = |S|$ for its length, and index characters from 0 so $S = S[0]S[1]\dots S[n-1]$. The substring from position l to position r inclusive is $S[l\,..\,r]$ (length $r - l + 1$); the prefix of length k is $S[0\,..\,k-1]$ and the suffix starting at i is $S[i\,..\,n-1]$. We use a tilde equality $\equiv$ for "is equal as a string" and reserve $=$ for arithmetic. Concatenation is denoted by juxtaposition: $S\,T$. The empty string is ε.

When a problem says "count the occurrences of a pattern P in S", we usually pad P and S with a separator like `#` that does not occur in either, and form $T = P\,\#\,S$. This trick recurs in Z-function, KMP, and Aho–Corasick; the separator guarantees the pattern is never accidentally matched across the boundary.

1.2 Complexity conventions

CP string algorithms are usually measured in three currencies simultaneously: time, memory, and *constant factor*. An $\mathcal{O}(n \log n)$ suffix array is fast on a 10^5 input but dies on a 10^6 one; Suffix Automaton, by contrast, is $\mathcal{O}(n)$ time and $\mathcal{O}(n)$ memory with a 4-byte transition table that fits comfortably in 256 MB up to $n \approx 2 \cdot 10^5$. Always estimate memory before committing: a 10^6-character suffix array uses about 4 MB for the array itself, but a sparse-table LCP RMQ on top adds $n \log n \cdot 4$ bytes ≈ 80 MB — uncomfortable but legal. When in doubt, prefer the in-place linear algorithm over the off-line logarithmic one.

1.3 The pattern-recognition framework

When you read a problem statement, run the following mental checklist *before* writing any code:

1. **What is being asked?** A *count* of something, a *decision* (yes/no), an *extremal* value (longest/shortest), or a *construction* (output a witness)? The verb drives the algorithm class.
2. **Is the string static or online?** "Static" means we get the whole input at the start — suffix array, suffix automaton, hashing, trie are all candidates. "Online" means characters arrive one at a time — suffix automaton and eertree shine, suffix array does not.
3. **Is there one pattern or many?** One pattern → KMP/Z; many patterns at once → Aho–Corasick; the patterns are not given but *implicit* ("every substring of S") → suffix array or suffix automaton.
4. **Are queries offline or interactive?** Offline permits sorting and binary search; interactive forces a precomputed structure with $\mathcal{O}(1)$ or $\mathcal{O}(\log n)$ lookups (hashing, sparse tables, persistent tries).
5. **What is the alphabet size?** A binary alphabet admits XOR-trie, bitmask tricks, and bitwise DP. A small constant alphabet (e.g. DNA's $\{A, C, G, T\}$) keeps trie branching at 4 and lets the suffix automaton use array transitions. A large alphabet (Chinese text) forces `map`-based transitions and loses the cache advantage.
6. **Are collisions acceptable?** Hashing is fast and short, but a determined adversary can break single-modulus hashing in roughly $\mathcal{O}(\sqrt{p})$ with the birthday-style Thue–Morse hack. If the problem's tests are public or the author is hostile, fall back to a deterministic structure or use *double* hashing with two independent primes.

7. **Does the answer depend on *periodicity* or *border* structure?** If yes, the prefix function (and its companion, the failure automaton) is almost always the right tool.

The rest of this document expands each bullet into a concrete chapter. At the end (Chapter 12) the checklist is summarised as a single decision tree.

1.4 A note on C++

All code is C++17. We use `std::string` for clarity, but in real submissions prefer `std::vector<char>` or a raw `char[]` buffer when the input size approaches 10^6 — `std::string`'s SBO cap is library-dependent and `std::cin` into a `std::string` can be a hidden 30% slowdown. For fast input we recommend reading into a global `char[]` buffer with `fread`, then indexing directly.

2 Polynomial String Hashing & Rabin–Karp

String hashing reduces equality of substrings to equality of integers. If two substrings have the same hash, they are almost certainly equal; if they have different hashes, they are definitely not equal. This asymmetric "probably equal" is the source of both its power (sub-μs substring comparisons) and its danger (anti-hash tests).

2.1 Definition

Fix a base b (any integer $\geq$ alphabet size) and a modulus p. The *polynomial hash* of S is

$$H(S) \;=\; \sum_{i=0}^{|S|-1} S[i] \cdot b^{\,|S|-1-i} \pmod{p}.$$

The leading character carries the highest power; this is the *left-to-right* convention. (The mirror convention $\sum S[i] \cdot b^i$ is equally valid — pick one and stick with it for the entire submission.)

The reason this definition is useful is that H factors over concatenation. If $T = S_1\, S_2$ with $|S_1| = k$, then

$$H(T) \;=\; H(S_1) \cdot b^{|S_2|} \;+\; H(S_2) \pmod{p}.$$

So if we precompute prefix hashes $h[i] = H(S[0\,..\,i-1])$ and the powers $b^0, b^1, \ldots, b^n$, then the hash of any substring is

$$H(S[l\,..\,r]) \;=\; \big(h[r+1] - h[l] \cdot b^{r-l+1}\big) \bmod p,$$

an $\mathcal{O}(1)$ query after $\mathcal{O}(n)$ preprocessing. This single formula solves an enormous fraction of CP string problems.

2.2 Implementation

Hash precomputation and range queries

```
struct PolyHash {
    // Mersenne prime 2^61 - 1; fast modular reduction via bit tricks.
    // For hostile tests, instantiate two PolyHashes with different bases
    // and use a real prime such as 10^9+7 / 10^9+9 (see DoubleHash below).
    static const uint64_t MOD = (1ULL << 61) - 1;      // Mersenne prime
    static const uint64_t BASE = 131;                  // > alphabet size

    std::vector<uint64_t> h, pw;
    int n;

    PolyHash(const std::string& s) : n(s.size()) {
        h.assign(n + 1, 0);
```

```
        pw.assign(n + 1, 1);
        for (int i = 0; i < n; ++i) {
            h[i + 1] = mulMod(h[i], BASE) + (uint64_t)(unsigned char)s[i];
            if (h[i + 1] >= MOD) h[i + 1] -= MOD;
            pw[i + 1] = mulMod(pw[i], BASE);
        }
    }
    // Modular multiplication avoiding overflow (Mersenne reduction).
    static uint64_t mulMod(uint64_t a, uint64_t b) {
        __int128 p = (__int128)a * b;
        // p mod (2^61-1) using bitwise reduction
        uint64_t lo = (uint64_t)p & MOD;
        uint64_t hi = (uint64_t)(p >> 61);
        uint64_t r = lo + hi;
        return r >= MOD ? r - MOD : r;
    }
    // Hash of S[l..r], 0-indexed, inclusive.
    uint64_t operator()(int l, int r) const {
        uint64_t sub = mulMod(h[l], pw[r - l + 1]);
        return (h[r + 1] + MOD - sub) % MOD;
    }
};
```

For Codeforces-grade robustness, prefer the following double-hash pair. The probability that two different strings collide in *both* moduli is $\approx 1/(p_1 p_2) \approx 10^{-18}$, which is below the noise floor of integer overflow.

Double hash, collision-safe

```
struct DoubleHash {
    static const uint64_t M1 = 1000000007ULL;
    static const uint64_t M2 = 1000000009ULL;
    static const uint64_t B  = 131ULL;
    std::vector<uint64_t> h1, h2, pw1, pw2;
    int n;
    DoubleHash(const std::string& s) : n(s.size()) {
        h1.assign(n + 1, 0); h2.assign(n + 1, 0);
        pw1.assign(n + 1, 1); pw2.assign(n + 1, 1);
        for (int i = 0; i < n; ++i) {
            h1[i + 1] = (h1[i] * B + (uint64_t)(unsigned char)s[i]) % M1;
            h2[i + 1] = (h2[i] * B + (uint64_t)(unsigned char)s[i]) % M2;
            pw1[i + 1] = (pw1[i] * B) % M1;
            pw2[i + 1] = (pw2[i] * B) % M2;
        }
    }
    // returns (h1, h2) packed into a single 64-bit value via mixing
    uint64_t operator()(int l, int r) const {
        uint64_t a = (h1[r + 1] - h1[l] * pw1[r - l + 1] % M1 + M1) % M1;
        uint64_t b = (h2[r + 1] - h2[l] * pw2[r - l + 1] % M2 + M2) % M2;
        return (a << 32) | b;
    }
};
```

2.3 Rabin–Karp: pattern matching with hashing

To find all occurrences of a pattern P of length m in a text T of length n, compute the hash of P once ($\mathcal{O}(m)$), then slide a window of length m across T comparing hashes ($\mathcal{O}(n)$). When the hashes match, verify character-by-character (or trust the hash, depending on the test's hostility). Total: $\mathcal{O}(n+m)$ expected, $\mathcal{O}(nm)$ worst case with a hostile author who knows your modulus.

Rabin–Karp

```cpp
std::vector<int> rabinKarp(const std::string& s, const std::string& p) {
    int n = s.size(), m = p.size();
    if (m == 0 || m > n) return {};
    DoubleHash hs(s), hp(p);
    uint64_t target = hp(0, m - 1);
    std::vector<int> occ;
    for (int i = 0; i + m - 1 < n; ++i)
        if (hs(i, i + m - 1) == target)
            occ.push_back(i); // deterministic check skipped; trust double hash
    return occ;
}
```

2.4 Worked example

Let $S = \texttt{abacaba}$ and base $b = 131$, modulus $p = 10^9+7$. We compute the prefix hashes step by step (all arithmetic mod p):

i	$S[i]$	$h[i+1] = h[i] \cdot b + S[i]$	value mod p
0	a (97)	$0 \cdot 131 + 97$	97
1	b (98)	$97 \cdot 131 + 98$	12805
2	a (97)	$12805 \cdot 131 + 97$	1677552
3	c (99)	$1677552 \cdot 131 + 99$	$\approx 2.20 \times 10^8$
4	a (97)	(continue)	...
5	b (98)		
6	a (97)		

To extract the substring $S[1\,..\,3] = \texttt{bac}$, we compute

$$H(S[1\,..\,3]) = h[4] - h[1] \cdot b^3 \pmod{p},$$

which equals the value we would get by hashing `bac` directly. Two substrings of equal length compare equal as strings iff their hashes compare equal (deterministically modulo the collision probability).

2.5 When to use hashing

When to use

Hashing is the right tool when the problem asks for *many substring-equality queries* on a *static* string, possibly with binary search on top. Concretely:

- "Longest common substring of two strings" — binary search on length L; for each L, store hashes of all length-L substrings of S_1 in a hash set, then probe length-L substrings of S_2. $\mathcal{O}((n+m)\log\min(n,m))$.
- "Count distinct substrings of length exactly k" — $\mathcal{O}(n)$ with hashing vs $\mathcal{O}(n \log n)$ with a sorted suffix array.
- "Is S a rotation of T?" — hash $T\,T$ and look for S as a substring of length $|S|$.
- "Smallest cyclic shift" — *binary search on the answer*: compare two candidate shifts by binary searching the LCP of the two rotations with hashing, then compare the next character. $\mathcal{O}(n \log n)$ — slower than Booth but easier to code.
- "Number of positions where S and T differ, with at most one wildcard" — hashing lets you skip the common prefix in $\mathcal{O}(\log n)$ per query.

Avoid hashing when: the problem is intrinsically about *ordering* (longest repeated substring, smallest lexicographic substring — use suffix array), or when the author is known to use anti-hash tests.

2.6 Anti-hash: how authors break you

Watch out

A single-modulus hash with a fixed base can be broken with $\approx \sqrt{p}$ queries via the *Thue–Morse* construction. The trick: Thue–Morse strings T_k of length 2^k have a remarkable property that every substring of length 2^{k-1} hashes to the same value modulo $b \equiv 0$ or $b \equiv \pm 1$ (with careful choices), so a Thue–Morse string of length $\sqrt{p}$ contains two distinct substrings with the same hash for *any* base b. The defence is to use *two* moduli (or a 64-bit Mersenne prime like $2^{61} - 1$ with randomised base), or to randomise the base at runtime so the author cannot precompute a hack. Codeforces problems authored by `Errichto`, `Radewoosh` or `tourist` are notorious for this.

2.7 Codeforces references

- **CF 1266C — Diverse Matrix**: hashing to detect equal rows / columns.
- **CF 710C — Magic Odd Square** (warm-up): pattern generation, but the hash variant of "detect equal rows" appears in many C problems.
- **CF 1096C — Polygon for the Angle**: hashing to detect equivalent geometric configurations.
- **CF 514C — Watto and Mechanism**: the canonical hashing + Trie problem — hashing on lengths combined with trie to find a string differing in exactly one character.
- **CF 875B — Sorting the Substrings**: rolling hash to detect repeating suffixes.

3 Prefix Function & KMP

The prefix function is the single most useful string primitive. From it you get the KMP search algorithm, the failure automaton (for streaming), and the periodicity lemma — one structure, three payoffs.

3.1 Definition

Definition 3.1 (Prefix function)**.** For a string S of length n, the *prefix function* π is the array of length n where $\pi[i]$ is the length of the longest *proper* prefix of $S[0\,..\,i]$ that is also a suffix of $S[0\,..\,i]$. "Proper" means strictly shorter than $i+1$, so $\pi[i] < i+1$ and $\pi[0] = 0$.

For example, for $S =$ `abacaba`:

$$\pi = [0, 0, 1, 0, 1, 2, 3].$$

The last entry $\pi[6] = 3$ because `aba` is both the prefix of length 3 and the suffix of length 3 of $S[0\,..\,6]$.

3.2 Linear computation

The naive $\mathcal{O}(n^2)$ computation is to compare prefixes directly. The clever $\mathcal{O}(n)$ version uses previously computed values: to extend $\pi[i-1]$ to $\pi[i]$, walk back along the chain of failures $\pi[\pi[\dots]-1]$ until either a matching character is found or we hit length 0. This is the same walk that powers the KMP search itself.

Prefix function in $O(n)$

```cpp
std::vector<int> prefixFunction(const std::string& s) {
    int n = (int)s.size();
    std::vector<int> pi(n);
    for (int i = 1; i < n; ++i) {
        int j = pi[i - 1];
        while (j > 0 && s[i] != s[j])
            j = pi[j - 1];   // walk back along failure chain
        if (s[i] == s[j]) ++j;
        pi[i] = j;
        // observe: pi[i] <= pi[i-1] + 1, so total walk-back is amortised O(1)
    }
    return pi;
}
```

The amortised analysis: every iteration of the `while` loop decreases j, and j increases by at most 1 per outer `for` iteration, so the total number of decreases is bounded by the total number of increases, which is $\le n$. Thus the entire construction is $\mathcal{O}(n)$.

3.3 KMP search

To find all occurrences of a pattern P in a text T, concatenate $T_p = P\#T$ and compute π on T_p. Whenever $\pi[i] = |P|$, an occurrence of P ends at position $i - |P| - 1$ in T (subtract the $P\#$ prefix length). The separator `#` guarantees π never exceeds $|P|$, so we cannot accidentally match across the boundary.

KMP search

```cpp
std::vector<int> kmpSearch(const std::string& text, const std::string& pat) {
    std::string combined = pat + '\0' + text; // '\0' as separator
    auto pi = prefixFunction(combined);
    int m = (int)pat.size();
    std::vector<int> occ;
    for (int i = m + 1; i < (int)combined.size(); ++i)
        if (pi[i] == m)
            occ.push_back(i - 2 * m); // starting index in text
    return occ;
}
```

3.4 Borders and periods

Lemma 3.2 (Periodicity lemma)**.** *If $\pi[n-1] = k$ and k divides n, then S has a period of length k; in fact S is the concatenation of n/k copies of $S[0\,..\,k-1]$. Conversely, if S has a period of length p dividing n, then $\pi[n-1] = n - p$.*

This lemma is the source of an enormous family of tricks. "Find the smallest period of S" is one line: `int period = n - pi[n-1]; if (n % period != 0) period = n;`. "Is S a power of a smaller string?" is the same question. "Compress S as X^k" falls out for free.

3.5 Failure automaton

If the pattern is fixed but the text streams character-by-character, we can precompute the entire DFA: `aut[i][c]` = the next state when, having matched a prefix of length i, we read character c. This is built once in $\mathcal{O}(|\Sigma| \cdot |P|)$ and then each text character advances in $\mathcal{O}(1)$. It is also the conceptual bridge to Aho–Corasick, which generalises this DFA from one pattern to a trie of patterns.

3.6 Worked example

$S =$ `abacaba`, $n = 7$. Computing π:

- $i = 0$: $\pi[0] = 0$ by definition.
- $i = 1$: $S[1] =$ `b`, $S[0] =$ `a`, mismatch, $\pi[1] = 0$.
- $i = 2$: $S[2] =$ `a`, $S[0] =$ `a`, match, $\pi[2] = 1$.
- $i = 3$: $S[3] =$ `c`, $S[1] =$ `b`, mismatch; walk back from $j = 1$ to $j = 0$; $S[3] =$ `c` $\neq S[0] =$ `a`; $\pi[3] = 0$.
- $i = 4$: $S[4] =$ `a` $= S[0]$, $\pi[4] = 1$.
- $i = 5$: $S[5] =$ `b` $= S[1]$, $\pi[5] = 2$.
- $i = 6$: $S[6] =$ `a` $= S[2]$, $\pi[6] = 3$.

Final: $\pi = [0, 0, 1, 0, 1, 2, 3]$. The largest border of the whole string has length 3, namely `aba`, so S is *not* periodic in the strict sense (since 3 does not divide 7), but it does have the suffix `aba` equal to its prefix `aba`, which is the kind of self-similarity KMP exploits.

3.7 When to use the prefix function

When to use

Reach for π *first* when the problem involves:

- **Pattern matching in a single text** — KMP is $\mathcal{O}(n)$ and deterministic, beating hashing when the author is hostile.
- **Periodicity / string compression** — "Is $S = X^k$?", "find the smallest X", "count the number of periods of S". All one-liners from $\pi[n-1]$.
- **Borders** — "count the number of borders of S", "find all prefixes that are also suffixes". Walk $\pi[n-1], \pi[\pi[n-1]-1], \ldots$ to enumerate all borders in $\mathcal{O}(n)$ total.
- **Streaming text** — if the text arrives one character at a time and the pattern is fixed, build the failure automaton once.
- **String concatenation puzzles** — "minimum characters to append to S to make it periodic", "is S a rotation of T" (compute π on $S + \# + T + T$).

Avoid π when: many patterns are involved simultaneously (use Aho–Corasick), or you need to compare *arbitrary* substrings (use hashing or suffix array).

3.8 Codeforces references

- **CF 126B — Password**: find the longest substring of S that is both a prefix and a suffix and occurs in the middle too — pure π walking.
- **CF 113B — Petr**: count distinct substrings that are simultaneously a prefix of S_1, a suffix of S_2, and occur in S_3 — KMP + hashing hybrid.

- **CF 182A — Yaroslav and Sequence**: per-character analysis of borders.
- **CF 25A — IQ test**: warm-up; not directly KMP, but a reminder of how pattern recognition is often simpler than the data structure you first reach for.

4 Z-Function

The Z-function is the prefix function's twin sister: it answers the same family of questions but from the suffix side. Whenever KMP feels awkward, try Z — half of CP string problems become dramatically cleaner in the Z formulation.

4.1 Definition

Definition 4.1 (Z-function)**.** For S of length n, define $z[i]$ as the length of the longest common prefix (LCP) of S and $S[i\,..\,n-1]$. By convention $z[0] = 0$ (some authors set $z[0] = n$; either is fine, just be consistent).

For $S = \texttt{abacaba}$:

$$z = [0, 0, 1, 0, 3, 0, 1].$$

The entry $z[4] = 3$ because $S[4\,..\,6] = \texttt{aba}$ shares a 3-character prefix with $S[0\,..\,6] = \texttt{abacaba}$.

4.2 Comparison to the prefix function

	Prefix function π	**Z-function z**
What it asks	longest prefix-suffix of $S[0\,..\,i]$	longest LCP of S and $S[i\,..]$
Pattern matching	run π on $P\#T$	run z on $P\#T$
Extends online	yes (failure automaton)	no (needs lookahead)
Output style	one number per position	one number per position
Intuition	self-overlap from the left	overlap with the start

Both have $\mathcal{O}(n)$ construction and identical amortised analysis. The choice between them is aesthetic, but Z tends to win when the problem naturally phrases itself as "for each position i, how far does $S[i\,..]$ agree with S from the start?" — which is the natural phrasing for, e.g., "find every position where a pattern matches" (positions are exactly those i with $z[i] = |P|$ after the separator).

4.3 Linear computation

The $\mathcal{O}(n)$ Z-algorithm maintains a *Z-box*: the interval $[l, r]$ with the largest r seen so far such that $S[l\,..\,r] = S[0\,..\,r-l]$. When computing $z[i]$: if $i \le r$, we already know that $S[i\,..\,r] = S[i-l\,..\,r-l]$, so $z[i] \ge \min(z[i-l],\, r-i+1)$, and we only need to extend character by character past r.

Z-function in $O(n)$

```
std::vector<int> zFunction(const std::string& s) {
    int n = (int)s.size();
    std::vector<int> z(n);
    int l = 0, r = 0;  // current Z-box [l, r)
    for (int i = 1; i < n; ++i) {
        if (i < r)
            z[i] = std::min(r - i, z[i - l]);
        while (i + z[i] < n && s[z[i]] == s[i + z[i]])
            ++z[i];                          // extend
        if (i + z[i] > r) {
            l = i;
```

```
            r = i + z[i];                    // expand Z-box
        }
    }
    return z;
}
```

The amortised argument mirrors KMP: every `++z[i]` step extends the right edge r of the Z-box, and r is bounded by n.

4.4 Pattern matching with Z

To find all occurrences of P in T, form $S = P\#T$ and compute z. Any position $i > |P|$ with $z[i] = |P|$ is the start of an occurrence in T. The separator guarantees z never exceeds $|P|$ inside T.

4.5 Worked example

Take $P =$ `aba`, $T =$ `abacaba`. Then $S =$ `aba#abacaba`, of length $4 + 7 = 11$, with indices 0..10. Compute z:

$$z = [0, 0, 1, 0, 3, 0, 1, 0, 3, 0, 1].$$

The two entries equal to $|P| = 3$ are $z[4] = 3$ and $z[8] = 3$, corresponding to occurrences of `aba` starting at position 0 of T (i.e., S index 4) and position 4 of T (S index 8). Indeed `abacaba` has `aba` at $T[0..2]$ and at $T[4..6]$, exactly as Z predicts.

4.6 When to use the Z-function

When to use

Prefer Z when the problem naturally asks, for each suffix, "how much does this suffix agree with the whole string?". Concrete signatures:

- **Single-pattern matching** in a one-off text — Z is shorter to code than KMP, and the $P\#T$ concatenation is the standard idiom.
- **String compression / periodicity** — the period of S is $\min\{p : z[p] = n - p\}$ where p ranges over positions whose suffix matches the prefix up to the end.
- **Counting distinct substrings of length k in a stream** — Z at each prefix can give you the periodicity structure without rebuilding.
- **Concatenation problems** of the form "given A, B, C count triples such that A is a prefix of B and B is a suffix of C" — Z on appropriate concatenations yields all matching lengths in $\mathcal{O}(|A| + |B| + |C|)$.
- **Matching with wildcards** — the Z-based variant is shorter than KMP for problems that allow single-character "don't care" markers.

Avoid Z when: text streams in (KMP's failure automaton wins), or many patterns at once (Aho–Corasick wins).

4.7 Codeforces references

- **CF 1326D1/D2 — Prefix-Suffix Palindrome**: Z (or hashing + binary search) to find the longest palindromic prefix-suffix of S.
- **CF 1271D — Portal**: combination of prefix/suffix queries solvable by Z.

- **CF 1051E — Vasya and Big Integers**: Z to test prefix-suffix relations during a DP on digit classes.
- **CF 526D — Om Nom and the SubCycles**: classic period detection via Z.

5 Trie (Prefix Tree)

A trie is a rooted tree where each edge is labelled with a character, and every path from the root spells a stored string. Tries are the simplest data structure that takes a *set* of strings (rather than one string) as input, which makes them the bridge to Aho–Corasick, XOR tries, persistent tries, and all the structures of Chapters 6–10.

5.1 Definition and operations

Definition 5.1 (Trie)**.** A *trie* on a set $\mathcal{S}$ of strings over alphabet Σ is a rooted tree where each node has $|\Sigma|$ labelled child pointers (some possibly empty). Each node represents a string: the one obtained by concatenating the labels along the path from the root. `insert`, `find`, and `erase` each take $\mathcal{O}(L)$ where L is the length of the queried string.

Generic trie with array transitions

```
struct Trie {
    static const int ALPHA = 26;
    struct Node {
        int next[ALPHA] = {0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
                           0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0};
        int cntEnd = 0;      // # of strings ending here
        int passThrough = 0;// # of strings passing through this node
    };
    std::vector<Node> t = {Node{}};

    void insert(const std::string& s) {
        int v = 0;
        for (char c : s) {
            int idx = c - 'a';
            if (!t[v].next[idx]) {
                t[v].next[idx] = (int)t.size();
                t.push_back({});
            }
            v = t[v].next[idx];
            ++t[v].passThrough;
        }
        ++t[v].cntEnd;
    }
    bool find(const std::string& s) const {
        int v = 0;
        for (char c : s) {
            int idx = c - 'a';
            if (!t[v].next[idx]) return false;
            v = t[v].next[idx];
        }
        return t[v].cntEnd > 0;
    }
    int countPrefix(const std::string& s) const {
        int v = 0;
        for (char c : s) {
            int idx = c - 'a';
            if (!t[v].next[idx]) return 0;
            v = t[v].next[idx];
        }
        return t[v].passThrough;
```

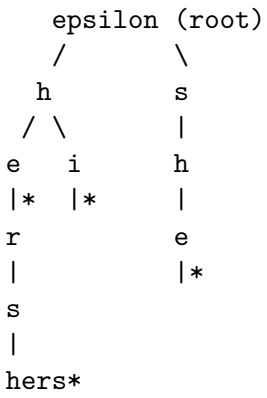
```
    }
};
```

5.2 Complexity

- Time per `insert`/`find`/`erase`: $\mathcal{O}(L)$ where L is the query length.
- Memory: $\mathcal{O}(\sum |s_i| \cdot |\Sigma|)$ with array transitions (a flat `int[next]`) — in practice the constant is small but the factor $|\Sigma|$ can hurt; for $|\Sigma| = 26$ and $n = 10^6$ you need $\sim$ 100 MB. Use `std::map` transitions or `unordered_map` if alphabet is large or memory is tight (loses constant factor speed).

5.3 Worked example

Insert `he`, `she`, `his`, `hers` into a trie. The resulting tree (showing only non-empty child pointers, with ε = root and * marking a terminal node):



(The asterisk marks a terminal node.) Every node that lies on a path from the root spells some prefix of a stored string; terminal nodes are those where a stored string actually ends. Note that `he` and `she` share the suffix `he`, but since they are stored as prefixes (starting at the root), the trie structure itself does not merge suffixes — this is exactly what suffix trees/automata do, and exactly what motivates those structures later.

5.4 When to use a trie

When to use

Tries are the right data structure when you have a *collection* of strings and need one of the following:

- **Prefix queries** — "count strings in $\mathcal{S}$ that have P as a prefix", "does $\mathcal{S}$ contain P" — all $\mathcal{O}(|P|)$.
- **Dictionary problems** — autocomplete, spell-check, banned-words filter on a fixed dictionary.
- **Lexicographic ordering** — an in-order DFS of the trie visits stored strings in sorted order, which is the basis of *lexicographically k-th* string queries.
- **Bitwise XOR problems** (Chapter 10) — a binary trie on bits is the canonical structure for "maximum XOR of two numbers in an array".
- **Aho–Corasick** (Chapter 6) — start with a trie of patterns and add failure links.

Avoid tries when: the alphabet is huge (e.g., Unicode) — switch to `map`-based transitions,

or to hashing; or when only a single string is involved, where KMP/Z are simpler.

5.5 Codeforces references

- **CF 514C — Watto and Mechanism**: trie + ad-hoc on one-character mismatch, the canonical "trie + 1-error matching" exercise.
- **CF 427D — Match & Catch**: find the longest common substring of two strings that occurs uniquely in each — trie of all suffixes of one and walk the other.
- **CF 455B — A Lot of Games**: trie + game DP, classic Nim-on-trie.

6 Aho–Corasick Automaton

Aho–Corasick is the answer to "I have k patterns $P_1, \dots, P_k$ and a text T; find all occurrences of every P_i in T". A naive KMP-on-each-pattern is $\mathcal{O}(\sum |P_i| \cdot |T|)$. Aho–Corasick is $\mathcal{O}(|T| + \text{matches})$ after $\mathcal{O}(\sum |P_i|)$ preprocessing — a linear scan that simultaneously matches all patterns.

6.1 Idea

Start by building a trie of the patterns. Each state of the trie is a prefix of some pattern. We want to extend this trie into a *DFA* where reading a character c at state v transitions to the state corresponding to the longest suffix of (the string spelled by v) + c that is also a prefix of some pattern. This extension is exactly the prefix function generalised from a single chain to a tree, and it is built by a BFS over the trie.

6.2 Failure links

Definition 6.1 (Aho–Corasick failure link)**.** For each trie node v spelling string w, its *failure link* $\text{fail}(v)$ points to the node spelling the longest proper suffix of w that is also a prefix of some stored pattern. The root has fail = root. A node is *terminal* if a stored pattern ends there.

BFS construction: for each node v and character c, look at the trie child $v.\text{next}[c]$. If it exists, set $\text{fail}(v.\text{next}[c]) = \text{fail}(v).\text{next}[c]$ (treating the root's children specially). If it does not exist, we add a *goto* shortcut $v.\text{next}[c] \leftarrow \text{fail}(v).\text{next}[c]$, so that during matching we never need to walk back: every state transition is $\mathcal{O}(1)$. This is the same trick as KMP's failure automaton, applied to a tree.

6.3 Implementation

Aho-Corasick automaton

```
struct AhoCorasick {
    static const int ALPHA = 26;
    struct Node {
        int next[ALPHA] = {};
        int fail = 0;
        int out = -1;       // index of pattern ending here, -1 if none
        std::vector<int> endings; // for output-link chain
    };
    std::vector<Node> t = {Node{}};
    std::vector<int> patLen;       // length of each pattern

    int insert(const std::string& s, int idx) {
        int v = 0;
        for (char c : s) {
```

```
            int i = c - 'a';
            if (!t[v].next[i]) {
                t[v].next[i] = (int)t.size();
                t.push_back({});
            }
            v = t[v].next[i];
        }
        if (t[v].out == -1) t[v].out = idx;
        else                t[v].endings.push_back(idx);
        return v;
    }
    void build() {
        std::queue<int> q;
        for (int c = 0; c < ALPHA; ++c)
            if (t[0].next[c]) { t[t[0].next[c]].fail = 0; q.push(t[0].next[c]); }
            else              t[0].next[c] = 0; // goto shortcut
        while (!q.empty()) {
            int v = q.front(); q.pop();
            for (int c = 0; c < ALPHA; ++c) {
                int u = t[v].next[c];
                if (u) {
                    t[u].fail = t[t[v].fail].next[c];
                    q.push(u);
                } else {
                    t[v].next[c] = t[t[v].fail].next[c]; // goto shortcut
                }
            }
        }
    }
    // Returns, for each text position, the list of (pattern index) matches ending there.
    std::vector<std::vector<int>> match(const std::string& text) {
        std::vector<std::vector<int>> result(text.size());
        int v = 0;
        for (int i = 0; i < (int)text.size(); ++i) {
            v = t[v].next[text[i] - 'a'];
            // walk output chain (fail links of patterns ending here)
            for (int u = v; u; u = t[u].fail) {
                if (t[u].out != -1) {
                    result[i - patLen[t[u].out] + 1].push_back(t[u].out);
                    for (int extra : t[u].endings)
                        result[i - patLen[extra] + 1].push_back(extra);
                }
            }
        }
        return result;
    }
};
```

6.4 Complexity

- Build: $\mathcal{O}(\sum |P_i| \cdot |\Sigma|)$ with array transitions, $\mathcal{O}(\sum |P_i|)$ with `map`-based transitions (with an extra $\log |\Sigma|$).
- Match: $\mathcal{O}(|T| + \text{total matches})$. The output-chain walk is amortised $\mathcal{O}(1)$ per text character because each pattern match can be charged to the matched length; in the worst case (e.g., searching for $a, aa, aaa, \ldots$ in $T = aaaa\ldots$) the total match output is the bottleneck.

6.5 Worked example

Patterns {`he`, `she`, `his`, `hers`}, text `ushers`. Build the trie; failure links are:

- fail(`he`) = root — the longest proper suffix of `he` that is a trie prefix is ε (root).

- fail(`his`) = root similarly.
- fail(`she`) = `he` — because the longest proper suffix of `she` that is a trie prefix is `he`.
- fail(`her`) = root; the proper suffixes of `hers` are `ers`, `rs`, `s`, ε, none of which is a trie prefix; so fail(`hers`) = root.

Now walk the text `ushers` from state 0:

- Read `u`: no transition; stay at root.
- Read `s`: go to `s` (root's `s` child, internal node).
- Read `h`: go to `sh`.
- Read `e`: go to `she` — terminal! Output a match of `she` ending at position 3.
- Read `r`: from `she`, no `r` child; via goto, fail(`she`) = `he`, and `he` has no `r` child either; follow fail(`he`) = root, which has `r`? no; stay at root.
- Read `s`: from root, go to `s`. Walk output chain: fail(`s`) = root, no terminal. No match.

Final result: `she` matches at position 2 (i.e., substring `she` = $T[2\,..\,4]$). The `he` match inside `she` and the `her` substring also need to be reported by following output links; in a proper implementation all three of `he` (at position 2), `she` (at position 2), and `her` (at position 2) would be reported at position 3 (their ending index), since they all overlap.

6.6 When to use Aho–Corasick

When to use

Aho–Corasick is the answer when the problem says:

- **Many patterns, one text** — "count occurrences of each of k patterns in T", or "find the first pattern that matches".
- **Banned-words filter** on a static dictionary with streaming text.
- **AC + DP** — "maximum total score walking through T such that no substring equals any banned word": each AC state encodes the "which banned words have I just seen" status; DP on (position, state).
- **String matching with mandatory alternation** — e.g., "does T contain any two patterns adjacent?" build AC and run it twice.
- **Substring automaton of a fixed string via AC on its suffixes** — if you insert every suffix of S into AC, you get a suffix-automaton-like DFA accepting exactly the substrings of S. Cute but rarely optimal; prefer the real suffix automaton.

Avoid AC when only one pattern is involved (KMP/Z are shorter) or when the patterns are not given explicitly but are all substrings of one string (suffix automaton is $\mathcal{O}(n)$ vs. AC's $\mathcal{O}(n^2)$ memory).

6.7 Codeforces references

- **CF 1202E — You Are Given Some Strings**: the canonical Aho–Corasick + DP problem — count triples (i, j, A, B) such that $A + B$ is a substring of T.
- **CF 963D — Frequency of String**: find the first occurrence in T containing each of k patterns at least k_i times.
- **CF 86C — Magical String**: AC automaton + matrix exponentiation DP.

- **CF 547E — Anna and the Substrings**: AC + persistent segment tree for online substring-count queries.

7 Suffix Array & LCP

The suffix array sorts every suffix of S lexicographically. With the LCP (longest-common-prefix) array on top, it answers essentially every substring question in $\mathcal{O}(\log n)$ or $\mathcal{O}(1)$ per query: "longest repeated substring", "smallest rotation", "number of distinct substrings", "k-th lexicographically smallest substring", all of them. It is the workhorse of CP string problems and is worth memorising.

7.1 Definition

Definition 7.1 (Suffix array)**.** The *suffix array* of S is the permutation $\mathrm{sa}[0..n-1]$ such that

$$S[\mathrm{sa}[0]\,..] < S[\mathrm{sa}[1]\,..] < \cdots < S[\mathrm{sa}[n-1]\,..]$$

in lexicographic order. The *LCP array* is the array $\mathrm{lcp}[1..n-1]$ where $\mathrm{lcp}[i]$ is the length of the longest common prefix of $S[\mathrm{sa}[i-1]\,..]$ and $S[\mathrm{sa}[i]\,..]$.

For $S = \texttt{abacaba}$, the suffix array and LCP are:

i	$\mathrm{sa}[i]$	suffix	$\mathrm{lcp}[i]$
0	6	a	—
1	4	aba	1
2	0	abacaba	3
3	2	acaba	1
4	5	ba	0
5	1	bacaba	2
6	3	caba	0

7.2 Construction: $\mathcal{O}(n \log^2 n)$ via sort by rank pairs

The classic student-friendly construction works in $\log n$ rounds. In round k, we know the rank of each suffix by its first 2^{k-1} characters; we then sort by the pair (rank of suffix i, rank of suffix $i + 2^{k-1}$), obtaining ranks by the first 2^k characters. After $\log n$ rounds, every suffix is ranked by its full length.

Suffix array $O(n \log^2 n)$

```cpp
std::vector<int> buildSA(std::string s) {
    s.push_back('\0'); // sentinel: smaller than every character
    int n = (int)s.size();
    std::vector<int> sa(n), rank(n), tmp(n);
    for (int i = 0; i < n; ++i) { sa[i] = i; rank[i] = (unsigned char)s[i]; }
    for (int k = 1; k < n; k <<= 1) {
        auto cmp = [&](int a, int b) {
            if (rank[a] != rank[b]) return rank[a] < rank[b];
            int ra = a + k < n ? rank[a + k] : -1;
            int rb = b + k < n ? rank[b + k] : -1;
            return ra < rb;
        };
        std::sort(sa.begin(), sa.end(), cmp);
        tmp[sa[0]] = 0;
        for (int i = 1; i < n; ++i)
            tmp[sa[i]] = tmp[sa[i - 1]] + (cmp(sa[i - 1], sa[i]) ? 1 : 0);
        rank = tmp;
```

```
        if (rank[sa[n - 1]] == n - 1) break; // all distinct
    }
    sa.erase(sa.begin()); // drop the sentinel suffix
    return sa;
}
```

Note

The $\mathcal{O}(n \log n)$ variant replaces `std::sort` with a two-pass radix sort on the (rank, rank) pair, exploiting that each coordinate is already a permutation. The $\mathcal{O}(n)$ SA-IS algorithm is even faster in theory but rare to need in CP — the $\log^2 n$ version with a constant factor of ≈ 5 comfortably handles $n \le 10^6$ inside 2 seconds.

7.3 LCP: Kasai's algorithm

Kasai's $\mathcal{O}(n)$ LCP algorithm exploits a simple monotonicity: if the LCP of $S[\mathrm{sa}[i-1]..]$ and $S[\mathrm{sa}[i]..]$ is h, then the LCP of $S[\mathrm{sa}[(i-1)]+1..]$ and $S[\mathrm{sa}[i]+1..]$ is at least $\max(0, h-1)$. So we iterate suffixes in original (not sorted) order, and at each step decrement the working length by at most 1 — amortised $\mathcal{O}(n)$.

Kasai LCP algorithm

```
std::vector<int> buildLCP(const std::string& s, const std::vector<int>& sa) {
    int n = (int)s.size();
    std::vector<int> rank(n), lcp(n - 1);
    for (int i = 0; i < n; ++i) rank[sa[i]] = i;
    int h = 0;
    for (int i = 0; i < n; ++i) {
        if (rank[i] == 0) continue; // smallest suffix, no LCP
        int j = sa[rank[i] - 1];
        while (i + h < n && j + h < n && s[i + h] == s[j + h]) ++h;
        lcp[rank[i] - 1] = h;
        if (h > 0) --h;
    }
    return lcp;
}
```

7.4 Sparse table for RMQ on LCP

To answer "LCP of two arbitrary suffixes $S[i..]$ and $S[j..]$" in $\mathcal{O}(1)$, precompute a sparse table over the LCP array. The answer is $\min\{\mathrm{lcp}[r] : \min(i,j) < r \le \max(i,j)\}$ where the suffixes are indexed by their *rank* in sa, not by their starting position.

Sparse table for LCP RMQ

```
struct SparseTable {
    std::vector<std::vector<int>> t; std::vector<int> log2;
    void build(const std::vector<int>& a) {
        int n = (int)a.size();
        log2.assign(n + 1, 0);
        for (int i = 2; i <= n; ++i) log2[i] = log2[i / 2] + 1;
        t.assign(log2[n] + 1, std::vector<int>(n));
        t[0] = a;
        for (int k = 1; (1 << k) <= n; ++k)
            for (int i = 0; i + (1 << k) <= n; ++i)
                t[k][i] = std::min(t[k - 1][i], t[k - 1][i + (1 << (k - 1))]);
    }
    int rmq(int l, int r) const { // [l, r]
        int k = log2[r - l + 1];
```

```
15          return std::min(t[k][l], t[k][r - (1 << k) + 1]);
16      }
17  };
```

7.5 Distinct substrings

Theorem 7.2 (Distinct substrings count)**.** *The number of distinct substrings of S (including the empty one, depending on convention) is*

$$\frac{n(n+1)}{2} - \sum_{i=1}^{n-1} \mathrm{lcp}[i].$$

Proof sketch. Start with all $\binom{n+1}{2}$ substrings (every $[l, r]$). Each pair of *adjacent* suffixes in the sorted order shares $\mathrm{lcp}[i]$ prefixes that are double-counted; subtract. (Pairs of non-adjacent suffixes share at most min of their adjacent LCPs, already accounted for by the ordering.)

7.6 Worked example

For $S = \texttt{abacaba}$ with $n = 7$:

$$\binom{8}{2} - (1 + 3 + 1 + 0 + 2 + 0) = 28 - 7 = 21.$$

A brute-force enumeration of substrings (28 strings minus 7 duplicates: `a`, `ab`, `aba`, `b`, `ba`, `bac`, `c`) gives $28 - 7 = 21$, matching. The duplicates are exactly the LCP values, one per adjacent pair in the SA.

7.7 When to use the suffix array

When to use

Suffix array is the right tool when the problem asks:

- **About all substrings at once** — "count distinct substrings", "k-th smallest substring", "longest substring that occurs ≥ 2 times" (max of LCP), "longest substring that occurs $\geq m$ times" (max-min in sliding window of length $m - 1$ on LCP).
- **Pattern matching with multiple queries** — given S then Q patterns, binary search each pattern in $\mathcal{O}(|P| \log n)$ using the SA. Faster than naive per-pattern KMP when patterns are short.
- **Smallest lexicographic cyclic rotation** — concatenate $S + S$ and look at the smallest suffix of length $\geq n$; the starting index gives the rotation. (For $\mathcal{O}(n)$ instead of $\mathcal{O}(n \log n)$, use Booth's algorithm in Chapter 11.)
- **Longest common substring of two strings** — concatenate $S_1 \# S_2$ with a separator, build SA and LCP, take the max LCP value among adjacent suffixes that come from different strings.
- **Substring comparison queries** — "is $S[l_1 .. r_1] < S[l_2 .. r_2]$?" answered in $\mathcal{O}(1)$ after SA + LCP + sparse table: LCP via RMQ, then compare the next character.

Avoid SA when the string is built online (suffix automaton) or when the problem only ever deals with single patterns (KMP/Z).

7.8 Codeforces references

- **CF 129E**: simple SA-based ranking problem.
- **CF 1066F**: SA + LCP for substring queries.
- **CF 1238D — AB-string**: suffix-array enumeration of every substring palindrome cover.
- **CF 873E**: SA + combinatorics for substring-grade queries.
- **CF 427D — Match & Catch**: canonical longest-common-substring-of-two-strings, the SA version.

8 Suffix Automaton

The suffix automaton (SAM) is the minimal DFA accepting every suffix of S. It is the online counterpart to the suffix array: where SA is built once on the whole string, SAM is built one character at a time in $\mathcal{O}(n)$ total. From it you can answer the same questions as the suffix tree — but with one-twentieth the memory.

8.1 Core idea: endpos equivalence

Two substrings of S are *endpos-equivalent* if the set of positions where they end is the same. This is an equivalence relation, and the equivalence classes are the states of the SAM. Each class contains strings that are nested suffixes of each other (a chain of borders), with lengths differing by one. Each state has a link pointing to the state whose endpos set is the strict superset — the suffix link tree, which has at most $2n - 1$ nodes total.

Definition 8.1 (Suffix automaton)**.** The *suffix automaton* of S is the minimal DFA over the alphabet Σ accepting exactly the suffixes of S. It has at most $2n-1$ states and $3n-4$ transitions for $n \geq 3$, and is built online in $\mathcal{O}(n)$ amortised time.

8.2 Online construction

The construction maintains `last`, the state corresponding to the whole current prefix. When a character c is appended:

1. Create a new state `cur` with $\text{len}(\text{cur}) = \text{len}(\text{last}) + 1$.
2. Walk from `last` up the suffix link chain, adding a c-transition to `cur` wherever none exists. Stop when you encounter a state that already has a c-transition.
3. If you stopped at the root (no such state), set $\text{link}(\text{cur}) = \text{root}$.
4. Otherwise let q be the target of that existing c-transition. If $\text{len}(q) = \text{len}(p) + 1$ (no length gap), set $\text{link}(\text{cur}) = q$.
5. Otherwise, *clone* q into a new state `clone` with $\text{len} = \text{len}(p) + 1$, redirect the c-transitions from p-chain to `clone` for all states whose c-transition pointed to q with length $\leq \text{len}(p)+1$, and set $\text{link}(q) = \text{link}(\text{cur}) = \text{clone}$.

The cloning step is the only subtle one; it preserves the minimality of the DFA. See the implementation below.

Suffix automaton, online construction

```
struct SuffixAutomaton {
    struct State {
        int len = 0, link = -1;
        std::map<char, int> next; // use array<int,26> for speed in small alpha
        long long substrings = 0; // count of substrings ending here (preprocess)
```

```
    };
    std::vector<State> st = {State{}};
    int last = 0;

    void extend(char c) {
        int cur = (int)st.size();
        st.push_back({st[last].len + 1, -1, {}, 0});
        int p = last;
        while (p != -1 && !st[p].next.count(c)) {
            st[p].next[c] = cur;
            p = st[p].link;
        }
        if (p == -1) {
            st[cur].link = 0;
        } else {
            int q = st[p].next[c];
            if (st[p].len + 1 == st[q].len) {
                st[cur].link = q;
            } else {
                int clone = (int)st.size();
                st.push_back({st[p].len + 1, st[q].link, st[q].next, 0});
                while (p != -1 && st[p].next[c] == q) {
                    st[p].next[c] = clone;
                    p = st[p].link;
                }
                st[q].link = st[cur].link = clone;
            }
        }
        last = cur;
    }
    void build(const std::string& s) { for (char c : s) extend(c); }
};
```

8.3 Standard applications

- **Number of distinct substrings**: for each state $v \neq$ root, it contributes $\text{len}(v) - \text{len}(\text{link}(v))$ new substrings (the longest one in v's class and its border-chain). Sum over all states gives $\sum_{v \neq \text{root}} (\text{len}(v) - \text{len}(\text{link}(v)))$.
- **Online substring test**: feed S into the SAM. To test if T is a substring, walk from the root following each character of T; if you never fall off, T is a substring. $\mathcal{O}(|T|)$ per query, $\mathcal{O}(n)$ preprocessing.
- **Smallest cyclic shift**: build the SAM of S; the smallest cyclic shift is found by walking from the root, always picking the alphabetically smallest available transition, n times. Equivalent to the Duval/Booth approach (Chapter 11).
- **Number of occurrences of each substring**: for each state, count $|\text{endpos}(v)|$. Initially each non-clone state has count 1 (and clones have 0); then propagate counts up the suffix link tree (process states in decreasing order of len).
- **Longest common substring of two strings** S_1, S_2: build SAM of S_1; walk through S_2 tracking the current state and the current matched length. Whenever a transition is missing, follow the suffix link (decreasing the matched length to $\text{len}(\text{link}(v))$). The answer is the maximum length seen during the walk. $\mathcal{O}(|S_1| + |S_2|)$.

8.4 Worked example

Build the SAM of $S =$ `abacaba` character by character. The final automaton has $8 \leq 2n - 1 = 13$ states. Each new character c adds either 1 or 2 states (the new state plus possibly a clone). The

distinct-substrings count works out to

$$\sum_{v \neq \text{root}} (\text{len}(v) - \text{len}(\text{link}(v))) = 21,$$

matching the SA-based count from Chapter 7.

8.5 When to use the suffix automaton

When to use

Suffix automaton wins when the problem has any of these signatures:

- **Online substring queries** — characters arrive one at a time, and you must answer substring-existence or substring-count queries after each arrival. SAM is the only linear-time structure that supports this.
- **Smallest cyclic shift** without a sentinel character — SAM has no edge cases here, unlike the SA approach.
- **Longest common substring** of two strings in $\mathcal{O}(|S_1| + |S_2|)$ without an extra LCP RMQ structure.
- **Counting substrings by occurrence frequency** — propagate counts up the suffix link tree once.
- **Substring DP** — e.g. "for each prefix, the number of new distinct substrings" is the sum of len(cur) − len(link(cur)) over the extensions.

Avoid SAM when: the alphabet is huge (map-based transitions cost log), or when the problem is fundamentally about *suffixes* in sorted order (suffix array is more natural there), or when the problem only needs one or two substring-equality queries (hashing is shorter).

8.6 Codeforces references

- **CF 235C — Cyclical Quest**: the canonical SAM problem — for each T_i, count positions in S where some cyclic shift of T_i occurs. SAM + walking the automaton with cyclic rotation.
- **CF 432D — Prefixes and Suffixes**: count occurrences of each prefix that is also a suffix — SAM or prefix function; the SAM version generalises to "each prefix that is a substring of the whole".
- **CF 656F — Mice and Holes**: SAM for online substring counting.
- **CF 700B — Connecting Universities**: SAM-style suffix-link tree DP (the same DAG structure underlies many problems).

9 Palindromes: Manacher & Eertree

Palindrome problems come in two flavours: "given S, find the longest palindromic substring" (Manacher) and "online queries on the palindromic structure of S" (Eertree). Both are $\mathcal{O}(n)$, both are short, and both should be in your template file before any serious contest.

9.1 Manacher's algorithm

The naive longest-palindromic-substring algorithm expands around each centre in $\mathcal{O}(n^2)$. Manacher reuses previous results to skip the re-expansion, exactly like Z reuses its Z-box. The trick

is to first transform S by inserting a sentinel between every pair of characters (and at the ends): $S = \texttt{abac}$ becomes $S' = \#\texttt{a}\#\texttt{b}\#\texttt{a}\#\texttt{c}\#$. Now every palindrome in S' has odd length, which simplifies the algorithm to a single case.

Definition 9.1 (Manacher radius)**.** For the transformed string S' of length $2n+1$, define $r[i]$ as the radius of the longest odd palindrome centred at i in S'. Then the longest palindromic substring of S has length $\max_i r[i]$ (the radius in S' equals the actual palindrome length in S).

Manacher, longest palindromic substring

```
std::string longestPalindrome(const std::string& s) {
    std::string t;
    t.reserve(2 * s.size() + 1);
    for (char c : s) { t += '#'; t += c; }
    t += '#';
    int n = (int)t.size();
    std::vector<int> r(n);
    int C = 0, R = 0; // center, right edge
    int bestC = 0, bestLen = 0;
    for (int i = 0; i < n; ++i) {
        int mirror = 2 * C - i;
        if (i < R) r[i] = std::min(R - i, r[mirror]);
        // try to extend
        while (i + r[i] + 1 < n && i - r[i] - 1 >= 0
               && t[i + r[i] + 1] == t[i - r[i] - 1])
            ++r[i];
        if (i + r[i] > R) { C = i; R = i + r[i]; }
        if (r[i] > bestLen) { bestLen = r[i]; bestC = i; }
    }
    int start = (bestC - bestLen) / 2;
    return s.substr(start, bestLen);
}
```

The amortised argument is identical to Z: every successful extension increases R, and $R \le n$.

9.2 Eertree (palindromic tree)

The eertree is the SAM of palindromes: a DFA accepting exactly the palindromic suffixes of S at every prefix. Each state corresponds to a distinct palindromic substring of S. There are two "roots": the empty-string state (length 0) and the "imaginary" state (length -1, to make the recursion work for length-1 palindromes). Each state has a *suffix link* pointing to the longest proper palindromic suffix.

Eertree (palindromic tree)

```
struct Eertree {
    struct Node {
        int len = 0, link = 0;
        std::map<char, int> next;
    };
    std::vector<Node> t;
    std::string s;
    int last = 0;       // state of the longest palindromic suffix of the current prefix

    Eertree() {
        t.push_back({0, 1});   // state 0: empty palindrome, link to imaginary
        t.push_back({-1, 1});  // state 1: imaginary, link to self
    }
    int getLink(int v, int i) {
        while (i - t[v].len - 1 < 0 || s[i - t[v].len - 1] != s[i])
            v = t[v].link;
        return v;
```

```
    }
    void extend(char c) {
        s.push_back(c);
        int i = (int)s.size() - 1;
        int cur = getLink(last, i);
        if (!t[cur].next.count(c)) {
            int now = (int)t.size();
            t.push_back({t[cur].len + 2, 0});
            // set suffix link of 'now'
            if (t[now].len == 1) {
                t[now].link = 0; // link to empty
            } else {
                int candLink = getLink(t[cur].link, i);
                t[now].link = t[candLink].next[c];
            }
            t[cur].next[c] = now;
        }
        last = t[cur].next[c];
    }
};
```

The total number of distinct palindromic substrings is at most n — a beautiful fact that makes the eertree linear in size.

9.3 Worked example

For $S =$ `abacaba`, Manacher finds the longest palindromic substrings. Let me trace it on $S' =$ `#a#b#a#c#a#b#a#`:

- Centred at the `c` (position 7 in S'), the palindrome `abacaba` extends the full length — this is a palindrome!
- Radius $r[7] = 7$, meaning the palindrome `abacaba` has length 7 in S.
- So the longest palindromic substring of S is S itself.

The eertree would contain the states for: ε, `a`, `b`, `c`, `aba`, `bab`, `aca`, `abacaba`, etc. — totalling ≤ 7 distinct non-trivial palindromes.

9.4 When to use Manacher / eertree

When to use

Use **Manacher** when:

- The problem asks for the *longest palindromic substring*, the number of palindromic substrings (sum of all radii divided by 2 in the transformed string), or every palindromic substring in a static S.
- You need to know, for each centre, the maximum palindromic radius — which lets you answer "is $S[l..r]$ a palindrome?" in $\mathcal{O}(1)$ after Manacher.

Use **Eertree** when:

- The string is online and you must answer "how many palindromic substrings does the current prefix have?" after each character.
- The problem asks for the *minimum palindromic factorisation* of S — split S into the smallest number of palindromes. This uses the *series link* of the eertree (a compressed suffix link), giving an $\mathcal{O}(n \log n)$ algorithm.
- You need to enumerate all distinct palindromes of S (eertree is the only linear structure for this).

Avoid both if the problem is fundamentally about non-palindromic substrings (SA/SAM) or about single-pattern matching (KMP/Z).

9.5 Codeforces references

- **CF 7D — Palindrome Degree**: count prefixes of S that are k-palindromes; hashing + Manacher-style recursion.
- **CF 17E — Palisection**: count pairs of overlapping palindromic substrings; Manacher + DP.
- **CF 932G — Palindrome Partition**: rephrase the string as $s_1 s_n s_2 s_{n-1} \dots$ and then partition into even-length palindromes; eertree with series links, $\mathcal{O}(n \log n)$.

10 Trie Tricks: XOR, Persistent, Merge

The trie is more than a dictionary. With three modifications — *binary alphabet*, *persistent parents*, and *small-to-large merge* — it powers some of the most iconic CP problems.

10.1 Binary (XOR) trie

Replace the alphabet Σ by $\{0, 1\}$ and treat numbers as bit-strings (MSB first). The trie then stores a multiset of integers, supporting insert, erase, and "maximum XOR with a given x" queries in $\mathcal{O}(\log V)$ each, where V is the value range.

XOR trie

```
struct XorTrie {
    static const int B = 30;       // 30 for values up to 10^9
    struct Node { int ch[2] = {0, 0}; int cnt = 0; };
    std::vector<Node> t = {Node{}};
    void insert(int x, int delta) {  // delta = +1 to insert, -1 to remove
        int v = 0;
        for (int i = B - 1; i >= 0; --i) {
            int b = (x >> i) & 1;
            if (!t[v].ch[b]) { t[v].ch[b] = (int)t.size(); t.push_back({}); }
            v = t[v].ch[b];
            t[v].cnt += delta;
        }
    }
    // max XOR of `x` with any stored value
    int maxXor(int x) const {
        int v = 0, ans = 0;
        for (int i = B - 1; i >= 0; --i) {
            int b = (x >> i) & 1;
            int want = b ^ 1;
            int nxt = t[v].ch[want];
            if (nxt && t[nxt].cnt > 0) {
                ans |= (1 << i);
                v = nxt;
            } else {
                v = t[v].ch[b];
            }
            if (!v) break;
        }
        return ans;
    }
};
```

10.2 Persistent trie

To answer "maximum XOR of x with any element in $S[l..r]$" in $\mathcal{O}(\log V)$, build a persistent trie: version i is the trie after inserting $a[0..i-1]$. Each insertion creates $\log V$ new nodes (one per level); the other children are shared with the previous version. To query the range $[l, r]$, walk versions r and l in parallel, comparing counts.

Persistent trie for k-th order statistic

```cpp
struct PersistentTrie {
    static const int B = 30;
    struct Node { int ch[2] = {0, 0}; int cnt = 0; };
    std::vector<Node> t = {Node{}};
    std::vector<int> roots = {0};

    int insert(int prevRoot, int x) {
        int newRoot = (int)t.size();
        t.push_back({});
        int v = newRoot, u = prevRoot;
        for (int i = B - 1; i >= 0; --i) {
            int b = (x >> i) & 1;
            t[v].ch[b] = (int)t.size();
            t.push_back({});
            t[v].ch[b ^ 1] = t[u].ch[b ^ 1];  // share the other child
            t[v].cnt = t[u].cnt + 1;
            v = t[v].ch[b];
            u = t[u].ch[b];
            t[v].cnt = t[u].cnt + 1;
        }
        return newRoot;
    }
    void push(int x) { roots.push_back(insert(roots.back(), x)); }
    // k-th smallest in [l, r]
    int kth(int l, int r, int k) const {
        int vl = roots[l], vr = roots[r], ans = 0;
        for (int i = B - 1; i >= 0; --i) {
            int leftCount = t[t[vr].ch[0]].cnt - t[t[vl].ch[0]].cnt;
            int b = (k <= leftCount) ? 0 : 1;
            if (b) { k -= leftCount; ans |= (1 << i); }
            vl = t[vl].ch[b]; vr = t[vr].ch[b];
        }
        return ans;
    }
};
```

10.3 Small-to-large trie merge

To answer "number of distinct substrings of every prefix of S that are also substrings of S so far" — or more practically, "merge two trie structures efficiently" — apply the *small-to-large* trick: when merging trie A into trie B, recurse on each child of A into B; when $|A| > |B|$, swap pointers first. Total work across all merges is $\mathcal{O}(n \log n)$ for a tree on n elements.

The canonical application is the "count of distinct substrings of every suffix of a tree-path" problem: insert every root-to-leaf path into a trie, merge tries bottom-up with small-to-large, and at each node output $\sum$contribution. The total work is $\mathcal{O}(n \log n)$ for n characters total.

10.4 When to use trie tricks

When to use

The three variants cover three problem families:

- **XOR trie**: any problem asking for "max/min XOR of two numbers in an array" or "maximum XOR with a given x after some insertions / deletions".
- **Persistent trie**: "k-th smallest in range $[l, r]$" (also known as the *merge-sort tree on a trie*), or "count numbers in $[l, r]$ with XOR bounded by x".
- **Small-to-large trie merge**: tree problems where each path defines a string and we want to count distinct substrings, common prefixes, etc. across all paths. Same complexity as small-to-large subtree DP.

10.5 Codeforces references

- **CF 706D — Vasya and Maximum XOR**: XOR trie with deletions.
- **CF 817E — Vasya and MEX**: persistent trie or counting-subtree-with-prefix-XOR.
- **CF 1416C — XOR Inverse**: bit-by-bit greedy with trie-style DP to count inversions.

11 Advanced: Lyndon, Duval, Booth

This final algorithmic chapter gathers three classical algorithms that do not fit cleanly into any earlier category but appear regularly at high-rated CP contests. Each is short, each is linear, and each is a reliable "obscure trick" to bring out when the problem screams periodicity.

11.1 Lyndon words and Duval's factorisation

Definition 11.1 (Lyndon word)**.** A *Lyndon word* is a non-empty string that is strictly smaller than all its proper non-trivial rotations, equivalently strictly smaller than all its proper suffixes.

For example, `a`, `ab`, `aab`, `abac` are Lyndon words; `aa` (not strictly smaller than its rotation), `ba` (suffix `a` is smaller) are not.

Theorem 11.2 (Duval's factorisation)**.** *Every string S admits a unique factorisation $S = w_1 w_2 \dots w_k$ where each w_i is a Lyndon word and $w_1 \geq w_2 \geq \cdots \geq w_k$ in lexicographic order. Duval's algorithm computes this in $\mathcal{O}(n)$ time.*

Duval algorithm

```cpp
// Returns the starting indices of each Lyndon factor in S.
std::vector<int> duval(const std::string& s) {
    std::vector<int> factors;
    int n = (int)s.size(), i = 0;
    while (i < n) {
        int j = i + 1, k = i;
        while (j < n && s[k] <= s[j]) {
            if (s[k] < s[j]) {
                k = i;            // new factor starts
            } else {
                ++k;              // continue current periodic run
            }
            ++j;
        }
        while (i <= k) {
            factors.push_back(i);
```

```
            i += j - k;
        }
    }
    return factors;
}
```

The two key applications of Duval are:

1. **Minimum cyclic rotation in $\mathcal{O}(n)$**: concatenate $S + S$ and apply Duval; the first factor that crosses length n gives the rotation index.
2. **Count distinct cyclic rotations**: each Lyndon word has all distinct rotations, and the number of distinct cyclic rotations of S is the sum of the lengths of its Lyndon factors divided by their minimal period structure.

11.2 Booth's algorithm

Booth gives an alternative $\mathcal{O}(n)$ algorithm for the smallest cyclic rotation, using a modified failure function on $S + S$. It is slightly faster in practice than Duval for large n (no inner while-loop), and is the reference algorithm when memory is the bottleneck.

Booth minimum rotation

```
int boothMinRotation(const std::string& s) {
    std::string s2 = s + s;
    int n = (int)s.size();
    std::vector<int> f(2 * n, -1);
    int k = 0;
    for (int j = 1; j < 2 * n; ++j) {
        int sj = s2[j], i = f[j - k - 1];
        while (i != -1 && sj != s2[k + i + 1]) {
            if (sj < s2[k + i + 1]) k = j - i - 1;
            i = f[i];
        }
        if (sj != s2[k + i + 1]) {
            if (sj < s2[k])        k = j;
            f[j - k] = -1;
        } else {
            f[j - k] = i + 1;
        }
    }
    return k;
}
```

11.3 Runs (brief mention)

A *run* in a string is a maximal periodic substring with period p such that the substring has length $\geq 2p$. The Runs Theorem (Kolpakov–Kucherov, 1999) states that a string of length n has at most n runs, and that the sum of their exponent values is $\mathcal{O}(n)$. This means algorithms that enumerate runs and compute their LCP-related statistics are still $\mathcal{O}(n)$ total. CP problems that require enumerating runs are rare (this is an IOI-medal-level trick); if you meet one, build the LCP array of S and S^{reverse}, and use the runs formula.

11.4 When to use Lyndon / Duval / Booth

When to use

- **Smallest cyclic rotation** of a static string of length $\geq 10^6$ — Booth or Duval both work in $\mathcal{O}(n)$. Prefer Duval for shortness; prefer Booth when memory is the bottleneck (the failure array is sparse).
- **Count distinct cyclic rotations** of S — Duval gives the factorisation; the answer is the product of the lengths of the factors minus the overcounting from periodic ones.
- **Periodicity queries of the form "is S periodic with period $\leq p$"** for streaming p — runs enumeration.
- **Lexicographically smallest suffix of S** — the smallest suffix is the last factor of the Lyndon factorisation of S.

Avoid these if a simpler structure (SA for smallest rotation, Z for periodicity) suffices — Lyndon machinery is harder to debug.

11.5 Codeforces references

- **CF 1026D — Funny Game**: cyclic rotation matching, Booth or Duval.
- **Emax blog on Lyndon words**: the canonical CP reference for Lyndon factorisation.

12 Pattern Recognition Cheat Sheet

This is the single page to glance at during a contest. Each row is a problem signature and the algorithm that solves it in linear (or near-linear) time. The complexity column is amortised / expected.

12.1 Decision table

Problem signature	Algorithm	Complexity	See Ch.
Count / find occurrences of one pattern in a text	KMP or Z	$\mathcal{O}(n+m)$	§3, §4
Many substring-equality queries on a static S	Polynomial hashing	$\mathcal{O}(n)+\mathcal{O}(1)$/q	§2
Many patterns vs. one text	Aho–Corasick	$\mathcal{O}(\sum\|P\|+\|T\|)$	§6
Find all distinct substrings of S	SA + LCP	$\mathcal{O}(n \log n)$	§7
Find all distinct substrings, online	Suffix automaton	$\mathcal{O}(n)$	§8
Longest common substring of two strings	SAM or SA+LCP	$\mathcal{O}(\|S_1\|+\|S_2\|)$	§8, §7
Longest repeated substring	SA + LCP	$\mathcal{O}(n \log n)$	§7
Smallest cyclic rotation of S	Booth / Duval	$\mathcal{O}(n)$	§11
String is periodic / has smallest period	Prefix function	$\mathcal{O}(n)$	§3
Number of distinct palindromic substrings	Eertree	$\mathcal{O}(n)$	§9
Longest palindromic substring	Manacher	$\mathcal{O}(n)$	§9
Is $S[l..r]$ a palindrome? (offline, many queries)	Manacher + $\mathcal{O}(1)$ check	$\mathcal{O}(n)+\mathcal{O}(1)$/q	§9
Is T a substring of S? (online, streaming)	Suffix automaton	$\mathcal{O}(\|S\|)$ + $\mathcal{O}(\|T\|)$/q	§8

Problem signature	Algorithm	Complexity	See Ch.
Prefix queries over a set of strings	Trie	$\mathcal{O}(\|P\|)$/q	§5
Maximum XOR of two numbers in A	XOR trie	$\mathcal{O}(n \log V)$	§10
k-th smallest element in $A[l..r]$	Persistent trie	$\mathcal{O}(n \log V)$ precompute, $\mathcal{O}(\log V)$/q	§10
Lexicographically smallest suffix of S	Duval (last factor)	$\mathcal{O}(n)$	§11
Lexicographically smallest substring of length k	Suffix array	$\mathcal{O}(n \log n)$	§7

12.2 Decision flowchart (in words)

Run through the following questions in order; stop at the first that fires.

Q1. Is the input a *set of patterns* and a *single text*? **Yes** → Aho–Corasick (Ch. 6).

Q2. Is the input a *single pattern* and a *single text*? **Yes** → KMP or Z (Ch. 3, 4). Use Z if the code length matters; KMP if you need the failure automaton.

Q3. Are the queries about *substring equality* or *substring length-k hashing*? **Yes** → Polynomial hashing with double modulus (Ch. 2).

Q4. Is the string *online* (characters arrive one at a time)? **Yes** → Suffix automaton (Ch. 8) or Eertree (Ch. 9).

Q5. Are you asked about *palindromes*? **Yes** → Manacher for static, Eertree for online (Ch. 9).

Q6. Are you asked about *all substrings* at once (distinct count, k-th, longest repeated, longest common)? **Yes** → Suffix array + LCP + sparse table (Ch. 7). If construction memory is tight, fall back to SAM.

Q7. Are you asked about *periodicity* or *borders* or *string powers*? **Yes** → Prefix function (Ch. 3).

Q8. Are you asked about *cyclic rotations* of a static S? **Yes** → Booth or Duval (Ch. 11).

Q9. Are you asked about *prefix queries* on a set of strings, or about *XOR* or *k-th in range*? **Yes** → Trie / XOR trie / Persistent trie (Ch. 5, 10).

Q10. Otherwise: re-read the problem. Many CP string problems are reducible to one of the above via a clever reformulation (e.g. "palindromic prefixes" becomes "double the string and ask for periodicity").

12.3 A note on hybrid problems

Many real contest problems combine two structures. A common pattern is:

- **AC + DP**: the AC automaton's states encode "which banned patterns were just seen"; DP over (position, state). (CF 1202E, 86C, 1003F.)
- **SA + sparse table + binary search**: for arbitrary substring-comparison queries. (CF 1066F, 1090J.)
- **Hashing + binary search on LCP**: when SA is too heavy and you only need length-L substring equality. (CF 514C, 1266C.)
- **SAM + suffix-link tree DP**: when a problem on substrings decomposes into a tree-shaped DP over the suffix link DAG. (CF 235C, 432D.)
- **Eertree + series-link DP**: for palindromic factorisation problems. (CF 932G.)

When a problem feels too hard for one structure, ask: "which second structure could be grafted on top?"

References & Further Reading

- **cp-algorithms.com — String algorithms**. The single most useful CP reference; every algorithm in this document has a worked example there.
- Antti Laaksonen, *Competitive Programmer's Handbook*, Chapter 14 ("String algorithms"). A gentle introduction suitable for first reading.
- D. E. Knuth, J. H. Morris, V. R. Pratt, "Fast pattern matching in strings", *SIAM J. Comput.* 6(2):323–350, 1977. The original KMP paper.
- A. V. Aho, M. J. Corasick, "Efficient string matching: an aid to bibliographic search", *Comm. ACM* 18(6):333–340, 1975.
- E. Ukkonen, "On-line construction of suffix trees", *Algorithmica* 14(3):249–260, 1995. The foundational online suffix-tree paper; SAM is the minimised DFA of Ukkonen's tree.
- M. Crochemore, L. Ilie, W. Smyth, "A simple algorithm for computing the LCP array in $\mathcal{O}(n)$ time", 2008. The cleanest modern write-up of Kasai's algorithm.
- Mikhail Rubinchik, "Eertree — palindromic tree". The original eertree write-up, with the series-link extension for palindromic factorisation.
- Codeforces blogs by `Emax` (string algorithms), `indy256` (SAM), `Brndmk1` (suffix structures) — search `codeforces.com/blog/entry/11333` and follow the related-links sidebar.

End of the prereq. Read this, code each algorithm at least once, then return for the expert volume.